# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Architecture Design Assignment |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | Mahitha.S | **Reg. No.:** | 192521065 |
| **Date:** | 26-10-2026 | **Duration:** | 60 minutes |

**Code of Conduct**

*I _____Mahitha.S(192521065)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _Mahitha.S_

## Annexure A – Architecture Design Assignment

**Instructions to Students:**

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

## 4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

## 5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

## 6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

# SECURE DIGITAL EXAMINATION AND ACADEMIC INTEGRITY PLATFORM

Architecture Design Analysis Report

## Table of Contents


1. Introduction and Problem Context
2. System Requirements Analysis
   - 2.1 Objectives and Stakeholders
   - 2.2 Functional Requirements
   - 2.3 Non-Functional Requirements
3. Selection of Software Architecture
4. Software Architecture Diagram
5. Components and Their Interactions
   - 5.1 Component Responsibilities
   - 5.2 Examination Workflow
   - 5.3 Data and API Communication
6. Quality Attribute Analysis
7. Architectural Decisions and Trade-offs
8. Security, Privacy, Ethics and Incident Response
9. Testing, Containerization, CI/CD and Monitoring
10. Risks and Mitigations
11. Conclusion
12. References


## 1. Introduction and Problem Context

Online examinations require more than an online question page. The platform must establish who is taking an examination, verify eligibility, release the correct assessment at the authorized time, preserve answers reliably, support evaluation and retain trustworthy records. Remote assessment introduces risks including impersonation, unauthorized assistance, examination-content leakage, outages and unfair interpretation of monitoring signals.

The supplied problem statement calls for a Secure Digital Examination and Academic Integrity Platform covering scheduling, secure delivery, identity verification, submission, evaluation, audit records, suspicious-activity detection, scalable architecture, secure APIs, role-based access control (RBAC), automated testing, containerization, CI/CD, monitoring, incident response, privacy and ethics. This report translates those objectives into a proposed architecture.

Design assumptions: the platform is web-based and serves students, invigilators, administrators and evaluators. Institutional single sign-on (SSO) and multi-factor authentication (MFA) may be integrated if available. No particular cloud provider, biometric vendor or learning-management system is assumed.

# 2. System Requirements Analysis

## 2.1 Objectives and Stakeholders

The system aims to deliver examinations securely, reduce unauthorized access and impersonation, protect papers and submissions, retain auditable records, detect potentially suspicious activity, improve reliability, reduce administrative effort and strengthen trust. Security controls must be balanced with accessibility and fairness.

| Stakeholder | Needs / responsibilities | Architectural implications |
|---|---|---|
| Student | View eligible exams; authenticate; answer, save and submit; view authorized results. | Accessible portal, resilient autosave, receipt, privacy notices and appeal route. |
| Invigilator | Oversee assigned sessions, record incidents and review relevant flags. | Restricted interface, least-privilege access and documented review workflow. |
| Exam administrator | Create schedules, assign cohorts, manage windows and generate reports. | Administrative RBAC, validation, approvals and audit trail. |
| Evaluator | Access assigned submissions, grade work and finalize marks. | Scoped access, versioned grading and change history. |
| Institution / auditor | Verify exam actions and result changes are traceable. | Tamper-evident events, retention policy and export controls. |

## 2.2 Functional Requirements

The priorities below are proposed from the problem statement: P1 is essential for secure examination delivery; P2 is important supporting capability. Acceptance criteria are proposed verification evidence, not claims that the platform has already passed testing.

| ID | Requirement | Priority | Acceptance evidence / criterion |
|---|---|---|---|
| FR-01 | Create, update, publish and cancel schedules; define duration, time window and eligible cohort. | P1 | Invalid or conflicting configuration is rejected or explicitly confirmed. |
| FR-02 | Authenticate students and staff and enforce roles. | P1 | Unauthenticated requests and unauthorized role actions are denied. |
| FR-03 | Verify candidate eligibility and examination access window. | P1 | Only eligible candidates can access an open exam. |
| FR-04 | Deliver examination content securely. | P1 | Paper access is authorized; content is not exposed through public URLs. |
| FR-05 | Support answer entry, autosave, final submission and receipt. | P1 | Final submission returns a unique receipt and server timestamp; retries do not create conflicting submissions. |
| FR-06 | Evaluate submissions and publish results under authorized workflows. | P1 | Only assigned evaluators can grade; publication follows configured approval rules. |
| **ID** | **Requirement** | **Priority** | **Acceptance evidence / criterion** |
| FR-07 | Maintain trustworthy audit records. | P1 | Authentication, access, submission, grading and administrative changes generate attributable events. |
| FR-08 | Detect unusual signals and create human-review cases. | P1 | Flags contain timestamps and supporting metadata; a flag alone is not proof of misconduct. |
| FR-09 | Generate controlled reports. | P2 | Exports are scoped, access-controlled and logged. |

| | | | |
|---|---|---|---|
| FR-10 | Notify users about schedules, receipts and results. | P2 | Notification failure does not undo a saved submission. |
| FR-11 | Support appeals and authorized corrections. | P2 | Cases have an assigned reviewer, outcome, reason and traceable history. |
| FR-12 | Monitor service health and security events. | P1 | Critical failures and security events raise actionable alerts. |

### 2.3 Non-Functional Requirements

| Attribute | Requirement | Design response |
|---|---|---|
| Scalability | Handle concurrent candidates and synchronized start/submit peaks. | Stateless instances, load balancing, horizontal scaling, queues and load testing. |
| Security | Protect identities, papers, answers, grades and privileged actions. | TLS, RBAC, MFA/SSO, encryption, secrets management, validation and audit. |
| Performance | Keep login, paper access, autosave and submission responsive. | Indexed queries, bounded payloads, capacity tests and asynchronous non-critical work. |
| Reliability | Avoid lost or duplicated answers; preserve exam state. | Durable writes, idempotency keys, bounded retries, backups and restore tests. |
| Availability | Keep critical services usable during component failures where feasible. | Redundant instances, health checks, rollback, timeouts and recovery plans. |
| Maintainability | Allow safe changes to individual capabilities. | Domain boundaries, API contracts, automated tests, code review and observability. |
| Privacy and fairness | Collect proportionate evidence and treat candidates consistently. | Data minimization, retention limits, accessible alternatives and human review. |

Proposed service targets for validation: 99.9% monthly availability for core exam APIs; p95 ordinary API read latency below 500 ms under an agreed test load; p95 autosave acknowledgement below 1 second under that load; zero acknowledged final submissions lost in recovery tests; and a recovery point objective (RPO) of 5 minutes or better for transactional exam data. The assignment provides no workload measurements or service-level objectives, so these are initial targets—not measured results.

## 3. Selection of Software Architecture

The recommended style is a **hybrid architecture**: a layered client–server system at the boundary, domain-separated services for core examination capabilities, and event-driven processing for audit, notifications and integrity-review workflows. The platform can initially deploy a manageable number of containerized modules and separate them further only when scale, security boundaries or team ownership justify it.

The layered structure separates presentation, API/security, business logic and data access. Domain separation assigns responsibilities to identity/access, exam management, assessment delivery, submission, evaluation and integrity/audit. Event-driven processing lets background consumers work independently without blocking the candidate's submission path.

| Style | Strengths | Limitations / assessment |
|---|---|---|
| Layered / client-server | Clear separation and simple initial deployment. | A single large application may become tightly coupled and scale inefficiently. |
| Pure microservices | Independent scaling and releases; potential fault isolation. | Adds network failures, distributed tracing, data consistency and operational overhead. |

| | | |
|---|---|---|
| Event-driven | Decouples notifications, audit consumers and analytics. | Duplicate delivery and eventual consistency require idempotency and monitoring. |
| Serverless | Managed scaling for suitable background tasks. | Provider limits and cold starts may be unsuitable for every timed workflow. |
| Recommended hybrid | Combines clear layers, domain ownership, targeted scaling and asynchronous work. | Needs disciplined API/event contracts and observability; complexity should be kept proportional. |

### Why the hybrid architecture fits

**Security boundaries:** all public traffic enters through a protected API layer; internal services enforce their own resource authorization. Browsers cannot directly access data stores.

**Critical transactions:** eligibility, answer persistence and final-submission confirmation require authoritative responses and durable writes.

**Decoupled background work:** notifications, audit indexing, analytics and alert routing consume durable events. A slow notification provider should not block an accepted submission.

**Workload-specific scaling:** assessment delivery and submission can scale separately from reporting and administration.

**Integration and maintainability:** versioned APIs and adapters can connect institutional SSO, learning-management or student-information systems where available.

## 4. Software Architecture Diagram

Figure 1 shows the logical design. Portals are treated as untrusted clients; the edge/API layer is the controlled entry point; domain services enforce business rules; and restricted stores retain transactions, encrypted files, events and telemetry. The identity provider is an optional institutional integration.

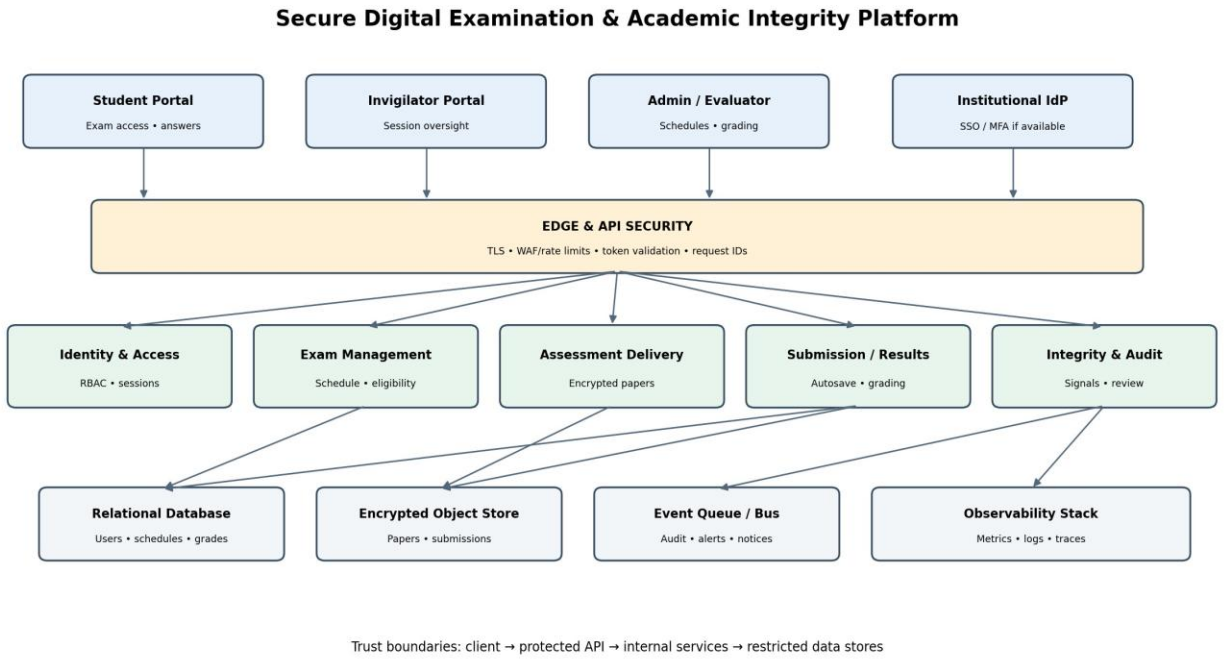



*Figure 1. Proposed logical architecture of the Secure Digital Examination and Academic Integrity Platform.*

The relational database stores structured transactional records such as schedules, eligibility and grades. Encrypted object storage holds papers and larger submission artefacts. The event bus carries events for audit, notification and analytics consumers. Monitoring receives operational telemetry and security alerts. Technology products are intentionally unspecified to avoid vendor dependence.

## 5. Components and Their Interactions

### 5.1 Component Responsibilities

**1. Student Portal**

Shows available exams, instructions, timer, answer interface, save status, receipt and authorized results. It should support keyboard navigation and low-bandwidth behavior. Eligibility and exam-window decisions are made server-side, not trusted to the browser.

**2. Invigilator Portal**

Shows assigned sessions and permitted oversight data; lets authorized personnel record incidents and review integrity flags. Access is limited to assigned sessions and the minimum data required.

**3. Administrator Portal**

Supports schedule creation, cohort assignment, examination configuration, role administration and reports. High-impact actions such as changing a live exam window or releasing results require explicit confirmation and an audit event.

**4. Edge and API Security Layer**

Terminates HTTPS, applies request-size and rate limits, validates tokens, routes requests and attaches correlation IDs. A web application firewall may filter common malicious traffic. Each service still validates authorization for its resources.

**5. Identity and Access Service**

Integrates with institutional SSO if available, applies MFA policy, manages sessions and maps identities to roles. Authentication proves control of an account; exam-specific eligibility is checked separately.

**6. Exam Management Service**

Creates schedules, time windows, cohort eligibility and exam states. It validates configuration and determines whether an exam is scheduled, open, closed or cancelled.

**7. Assessment Delivery Service**

Checks exam state and candidate eligibility before releasing paper content. It retrieves encrypted assets from restricted storage and avoids guessable public links.

**8. Submission Service**

Accepts autosaves and final submissions, validates exam state, persists answers and returns receipts. Idempotency keys prevent network retries from creating duplicate logical submissions. A receipt includes a unique identifier and server timestamp.

**9. Evaluation and Result Service**

Provides authorized access to submitted work, supports marking and review, records grade changes and controls result publication. Draft, approved and released marks remain distinct states.

**10. Integrity and Audit Service**

Collects security and examination events, identifies unusual patterns and creates review cases. Events record actor, action, object, time, outcome and correlation ID. A flag is not conclusive proof of misconduct.

**11. Relational Database**

Stores user-role mappings, schedules, exam state, submission metadata, grading state and release status. Transactions and constraints protect consistency; backups and restore tests protect recoverability.

### 12. Encrypted Object Store

Stores question papers, attachments and larger submission files. Server-side encryption, restricted service identities, access logging and retention rules protect content.

### 13. Event Queue / Bus

Carries durable events such as SubmissionAccepted, ExamScheduled, ResultPublished and IntegrityFlagRaised. Consumers retry transient failures, handle duplicates idempotently and route poison messages to a dead-letter queue.

### 14. Notification Service

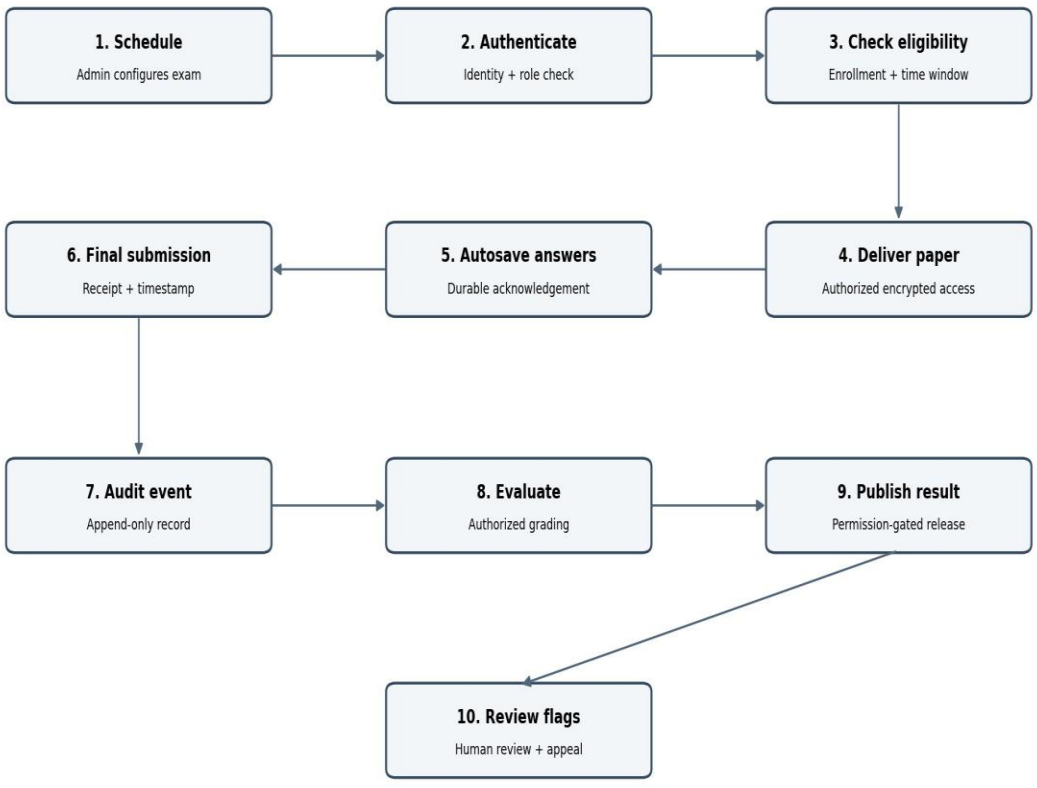
Sends approved email, SMS or in-platform notifications. Delivery attempts are recorded. Notification failure is separate from the core submission transaction.

### 15. Observability and Incident Response

Collects metrics, logs, traces and security events; detects latency, error rates, queue backlog and suspicious access; and routes actionable alerts. Logs exclude passwords, raw tokens and unnecessary answer content.

## 5.2 Examination Workflow

**Examination Lifecycle and Integrity Review Workflow**




**A suspicious signal triggers review; it does not by itself prove misconduct.**

*Figure 2. Examination lifecycle from scheduling to evaluation and integrity review.*

**Step 1 — Schedule:** an administrator configures the exam, cohort, time window, duration and assessment assets. The Exam Management Service validates and records the change.

**Step 2 — Authenticate and authorize:** the candidate signs in through the approved identity mechanism. The Identity and Access Service verifies the session and role; eligibility and timing are checked separately.

**Step 3 — Deliver:** the Assessment Delivery Service confirms the exam is open and the candidate is eligible, then retrieves the protected assessment asset.

**Step 4 — Save:** answer updates are validated and persisted. The service returns an acknowledgement; retries use idempotency keys or version identifiers.

**Step 5 — Submit:** the server checks the deadline and state, stores the final version, creates a receipt and emits SubmissionAccepted. Audit and notification consumers process the event asynchronously.

**Step 6 — Evaluate and publish:** authorized evaluators mark assigned submissions. Results are released only under the configured approval policy.

**Step 7 — Review:** unusual signals create review cases. An authorized reviewer considers context, records a decision and supports an appeal process.

### 5.3 Data and API Communication

Client-to-server communication uses HTTPS with TLS. Internal service communication should be authenticated and encrypted across trust boundaries. REST-style JSON APIs are suitable for synchronous operations; an event broker handles asynchronous work. API contracts should be versioned, documented and validated.

| Operation | Typical interaction | Important control |
|---|---|---|
| Sign in | Portal → Identity provider / Identity Service | MFA policy, secure sessions, rate limiting. |
| List eligible exams | Portal → API → Exam Management | Server-side role, enrollment and cohort checks. |
| Fetch paper | Portal → Assessment Delivery → Object Store | Exam-window check, authorization, no public object access. |
| Autosave answer | Portal → Submission Service → Database / Object Store | Validation, idempotency/versioning and durable acknowledgement. |
| Final submission | Portal → Submission Service → persistent record → event bus | Deadline enforcement, receipt, timestamp and integrity hash. |
| Raise review case | Event bus → Integrity/Audit consumer | Evidence minimization, reviewer authorization and traceability. |
| Publish result | Evaluator → Result Service → database + event | Role check, approval policy and audit event. |

Events should contain a unique event ID, event type, schema version, timestamp and correlation ID. Sensitive answer content should not be embedded if a reference or minimal metadata is sufficient. Transactional changes should be committed before success is returned. An outbox pattern—committing the business change and event record in one transaction—reduces the risk of a saved submission without its corresponding event. Consumers must still tolerate duplicate delivery.

Audit records should be append-only from the application's perspective, protected by restrictive permissions and monitored for deletion or alteration. Hash chaining, signed checkpoints or write-once storage can improve tamper evidence. Audit data remains subject to approved retention and privacy rules.

## 6. Quality Attribute Analysis

Figure 3 is a qualitative assessment of how directly the proposed design addresses the required quality attributes. It is not a benchmark and does not indicate that the platform has been implemented.

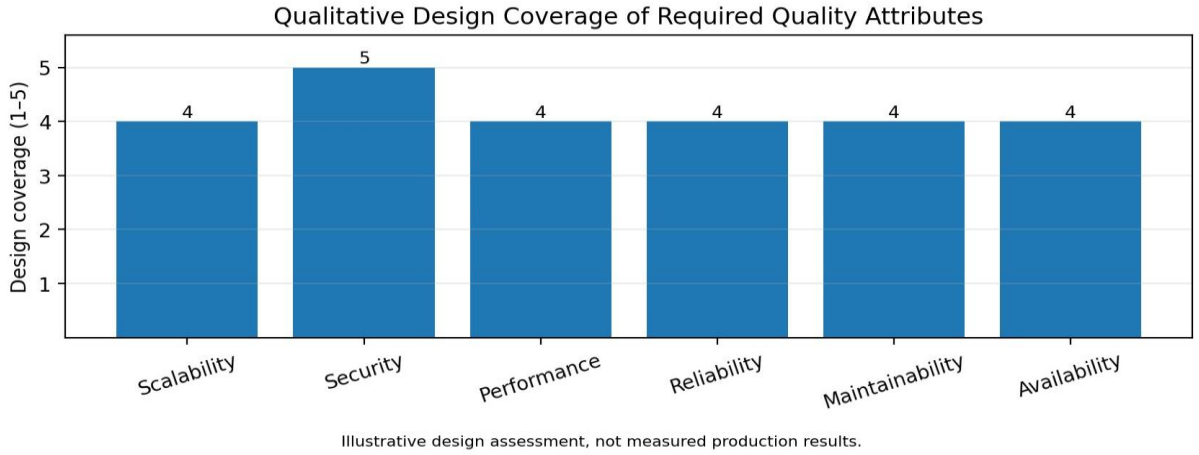



*Figure 3. Qualitative design coverage of required quality attributes (illustrative, not measured).*

| Attribute | Architectural mechanisms | Limitations / verification |
|---|---|---|
| Scalability | Stateless API instances, load balancing, horizontal scaling, queue buffering, indexes and connection pooling. | A shared database may bottleneck; test realistic concurrency and synchronized bursts. |
| Security | TLS, SSO/MFA, RBAC, least privilege, secret management, encryption, API validation and audit. | Architecture alone cannot guarantee security; perform threat modelling, scans and penetration tests. |
| Performance | Separate critical requests from background work; index hot queries; bound payloads; measure p95/p99 latency. | Caching sensitive or time-dependent state may leak data or become stale; verify under load. |
| Reliability | Durable commits, idempotent submissions, bounded retries, dead-letter handling, backups and health checks. | Retries may amplify overload; use backoff, limits and recovery procedures. |
| Maintainability | Domain boundaries, versioned API/event schemas, automated tests, code review and observability. | Too many services add operational burden; split only when justified. |
| Availability | Redundant instances, health-based routing, graceful degradation, rollback, timeouts and recovery plans. | Identity/database outages can still block exams; define and test RTO/RPO. |

### Performance and capacity reasoning

The workload is likely to be bursty: many candidates may sign in, retrieve papers or submit near the same scheduled time. Capacity tests should model concurrent logins, repeated autosaves, simultaneous final submissions and event consumption. Measure throughput, p95/p99 latency, errors, database saturation and queue backlog rather than average response time alone.

A simple planning estimate is **request rate = active candidates × average requests per candidate per second**. For an illustrative test scenario of 2,000 active candidates at 0.2 API requests per second each: 2,000 × 0.2 = **400 requests/second**. This is an example calculation, not a claim about actual enrollment or expected production load. Add a peak factor for synchronized activity and verify the resulting design through load tests.

### Reliability and recovery

Issue a submission acknowledgement only after the accepted answer state is durably stored. If the response is lost in transit, the client retries with the same idempotency key and obtains the same logical outcome. Encrypt

backups and test restoration. Recovery exercises should verify that exam state, submissions, grading records and audit events remain consistent. RTO and RPO must be agreed with the institution.

## 7. Architectural Decisions and Trade-offs

| Decision | Reason | Trade-off / mitigation |
|---|---|---|
| Hybrid layered, domain-separated architecture | Balances clear responsibilities, targeted scaling and manageable operations. | Boundaries require discipline; document ownership and avoid unnecessary service splitting. |
| Synchronous submission persistence | Candidates need authoritative confirmation that answers have been accepted. | Database latency affects acknowledgement; optimize writes and test peak load. Do not rely solely on an in-memory queue. |
| Asynchronous notifications and analytics | These tasks should not delay exam delivery or submission. | Events can be delayed or duplicated; use durable queues and idempotent consumers. |
| Relational DB + encrypted object store | Structured records need transactions; large files suit object storage. | Multiple stores complicate backup/access policy; define ownership and restore procedures. |
| RBAC plus resource-level authorization | Roles limit broad access; resource checks restrict access to assigned exams and submissions. | Policy logic is complex; test negative access cases. |
| Human review of integrity flags | Automated signals can be noisy or affected by connectivity, disability or environment. | Review takes staff time; use transparent triage and a documented appeal path. |
| Containerization and gated CI/CD | Repeatable builds and quality gates reduce release inconsistency. | Pipeline compromise is possible; protect signing keys and restrict deployment permissions. |

### Trade-off discussion

**Security versus usability:** stronger authentication reduces misuse but can create friction. Provide accessible MFA options and recovery procedures while retaining institutional security requirements.

**Consistency versus availability:** final submissions and grade approvals need authoritative state, so they use durable transactional writes. Some reports and notifications can be eventually consistent.

**Independence versus complexity:** separate services can scale independently but introduce network failures and deployment overhead. Begin with manageable domain modules and extract services when workload or ownership justifies it.

**Integrity monitoring versus privacy:** collecting more signals may increase detection but also intrusiveness and false positives. Prefer disclosed, proportionate signals, limited retention, restricted access and human review.

**Resilience versus cost:** redundancy, backups and monitoring increase operating cost. Prioritize critical delivery and submission paths according to institutional risk and service objectives.

### Future enhancements and integration

The design can integrate with a learning-management system for course and roster synchronization, student-information systems for enrollment, institutional identity federation, approved notification gateways, accessibility tools and analytics dashboards. Integrations should use scoped credentials, documented contracts and audit logging.

Future enhancements may include multi-region disaster recovery, status pages, accessibility accommodations, configurable alert triage and privacy-preserving analytics. Biometric or AI-based proctoring should not be

enabled by default; it requires separate assessment of necessity, proportionality, accuracy, bias, consent/legal basis and appeal mechanisms.

## 8. Security, Privacy, Ethics and Incident Response

### 8.1 Security controls

The design uses defence in depth. Authentication verifies account control, authorization checks the requested action and resource, encryption protects data in transit and at rest, and audit records support accountability. Secure development includes threat modelling, dependency management, secrets scanning, static analysis and penetration testing.

| Control area | Proposed control | Purpose |
|---|---|---|
| Identity | Institutional SSO if available; MFA for privileged roles; secure session lifecycle. | Reduce account compromise and unauthorized privileged access. |
| Authorization | RBAC plus resource-level checks; least privilege; separation of duties for result release. | Prevent access to other candidates' papers, submissions or grades. |
| Content protection | TLS, encryption at rest, restricted object access and short-lived authorized access where appropriate. | Reduce leakage of examination material and submissions. |
| API protection | Schema validation, output encoding, rate limits, request-size limits and CSRF protection where applicable. | Reduce common web/API attack paths. |
| Audit integrity | Append-only application behavior, restricted deletion, protected backups and tamper-evidence checks. | Support investigation and accountability. |
| Supply chain | Managed secrets, short-lived credentials, dependency scanning, signed images and restricted pipeline permissions. | Reduce credential leakage and compromised releases. |

### 8.2 Privacy and academic fairness

Collect only data needed to deliver the exam, protect integrity, resolve disputes and meet approved institutional obligations. Before monitoring is enabled, document what is collected, why it is necessary, who can access it, how long it is retained and how a student can raise a concern. Video, audio, device telemetry or biometric data should not be collected unless separately justified, lawfully governed and proportionate.

Integrity signals can be affected by unstable connectivity, assistive technologies, shared devices or environmental noise. A flag should trigger contextual review, not automatic punishment. Students should receive understandable rules and notice of applicable monitoring, a way to explain unusual events and a fair appeal process. Sensitive evidence should be accessible only to trained reviewers and access should be logged.

Retention should be purpose-based: keep assessment and audit records for the institutionally required period, then delete or anonymize them when no longer needed. Reports should not expose personal data to users who do not need it. Privacy and fairness are requirements throughout design, testing and operations.

### 8.3 Incident response

The incident process covers preparation, detection, triage, containment, recovery and post-incident review. Examples include account takeover, leaked papers, widespread failed submissions, queue backlog or unexpected grade changes.

| Phase | Example actions |
|---|---|
| Prepare | Define owners, severity levels, contacts, runbooks, backup/restore procedures and exam-day escalation roles. |

| | |
|---|---|
| Detect and triage | Correlate alerts with request IDs and audit events; identify affected exam, scope and severity. |
| Contain | Revoke compromised sessions, restrict affected endpoints, pause unsafe deployments and preserve evidence. |
| Recover | Restore trusted services, validate submission consistency, reconcile pending events and communicate status. |
| Learn | Document impact and root cause, update controls/runbooks, test corrective actions and review notification obligations. |

## 9. Testing, Containerization, CI/CD and Monitoring

Automated testing and delivery are essential architecture concerns: they help identify defects before exam day and make releases repeatable.

### 9.1 Testing strategy

| Test level | What to verify | Example acceptance check |
|---|---|---|
| Unit tests | Schedule rules, eligibility, deadline logic, access policies and grade transitions. | Ineligible candidates are denied; closed exams reject submissions unless policy allows them. |
| API / integration | Gateway, identity, database, object storage and event contracts. | Retrying with the same idempotency key creates one logical submission. |
| End-to-end | Candidate journey from login to result; admin and evaluator workflows. | Authorized users complete the flow; unauthorized roles cannot invoke protected actions. |
| Security tests | Broken access control, injection, session handling, secrets and rate limits. | Student A cannot fetch Student B's submission by changing an identifier. |
| Load / stress | Synchronized logins, paper fetches, autosaves, submissions and event consumers. | Agreed latency/error targets hold at target concurrency; overload fails safely. |
| Resilience tests | Database failover, queue outage, notification outage, instance loss and restore. | Notification outage does not lose a stored submission; recovery meets agreed objectives. |
| Accessibility / fairness | Keyboard use, screen-reader labels, low bandwidth, accommodations and review outcomes. | Critical flows support approved assistive workflows and flags can be appealed. |

### 9.2 CI/CD pipeline

The proposed pipeline is: developer commit → source and secret scanning → unit tests → container image build → dependency and image scan → integration tests → artifact signing and publication → staging deployment → end-to-end and smoke tests → production approval → progressive deployment → health verification and rollback if required. Production credentials must not be embedded in source code or container images.

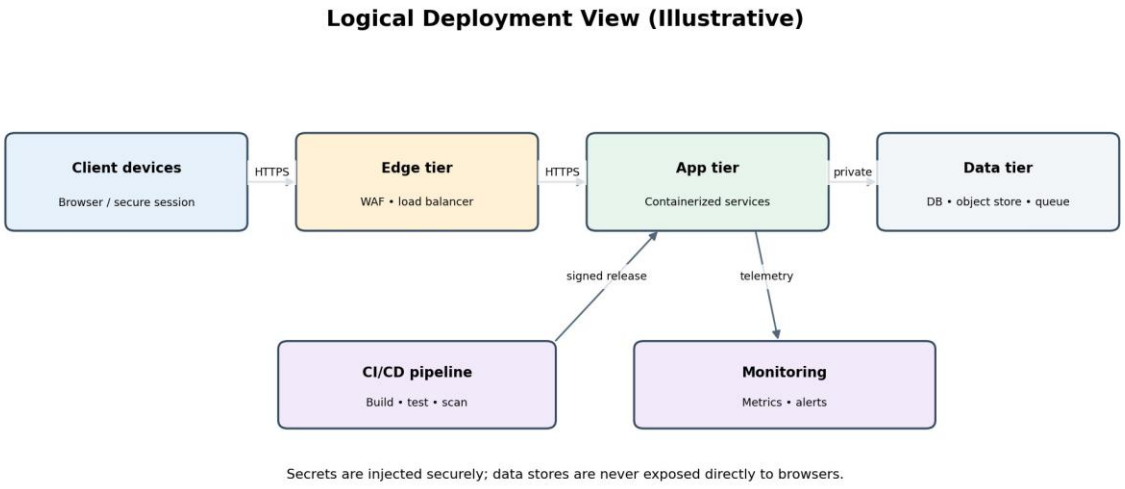



*Figure 4. Logical deployment view showing client, edge, application, data, CI/CD and monitoring tiers.*

Containerization packages a deployable application unit with its runtime and dependencies, improving consistency between test and production. Orchestration can be introduced when scale requires it, but it does not replace access control, backups, security or monitoring.

### 9.3 Monitoring and operational readiness

Monitor availability, request rate, latency percentiles, error rate, authentication failures, rejected authorization attempts, submission acknowledgements, database health, queue backlog, dead-letter events, storage access and deployment status. Alerts must be actionable and assigned to an owner. Logs should include correlation IDs but exclude passwords, raw access tokens and unnecessary answer content.

Before a major exam, verify capacity-test results, backup status, restore evidence, identity-provider availability, paper access controls, test submissions, incident contacts, status communications and rollback readiness. Review incidents and service metrics after the exam.

## 10. Risks and Mitigations

| Risk | Potential impact | Mitigation |
|---|---|---|
| Traffic spike at exam start/end | Slow login, delayed autosave or failed submission. | Load test bursts, scale application tier, queue non-critical work and monitor saturation. |
| Account takeover / impersonation | Unauthorized exam access or result changes. | MFA policy, secure sessions, rate limits, anomaly alerts and auditable actions. |
| Paper or answer leakage | Compromised confidentiality and fairness. | Restricted storage, least privilege, short-lived access, encryption and access logs. |
| Network interruption | Lost draft updates or uncertain submission status. | Autosave, idempotent retries, explicit save status, final receipt and recovery rules. |
| False-positive integrity flag | Unfair allegation or penalty. | Human review, contextual evidence, transparent policy, accommodations and appeal. |
| Database or region outage | Unavailable exam functions or delayed recovery. | Backups, tested restore, failover plan, agreed RTO/RPO and contingency policy. |

| | | |
|---|---|---|
| Pipeline/dependency compromise | Malicious code or vulnerable release. | Protected branches, scanning, signed artifacts, approval gates and rollback. |
| Excessive data retention | Privacy harm and larger breach impact. | Retention schedule, deletion/anonymization, access reviews and justified legal holds. |

## 11. Conclusion

This report proposes a hybrid, layered client–server architecture with domain-separated examination services and event-driven processing for non-critical background work. It addresses the stated objectives through secure scheduling and delivery, identity and role checks, protected submission, controlled evaluation, audit records, integrity review, automated testing, containerized deployment, CI/CD, monitoring and incident response.

The key design principle is to protect critical examination transactions without creating unnecessary complexity. Authorization is enforced server-side; final submissions are durably stored before confirmation; audit and notification tasks are decoupled; and suspicious activity is reviewed by authorized humans rather than treated as automatic proof of misconduct. Proposed performance and availability targets must be validated through realistic tests.

Privacy, accessibility and fairness are treated as system requirements. With data minimization, least-privilege access, transparent monitoring, appeal routes and tested recovery procedures, the platform can provide a stronger foundation for reliable, auditable and trustworthy digital examinations.

## Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **System Requirement Analysis (30%)** | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis. | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| **Selection of Software Architecture (25%)** | Selects the most appropriate architectural style with strong technical justification aligned to system requirements. | Selects a suitable architecture with reasonable justification. | Architecture is partially suitable with limited justification. | Inappropriate architecture selected or justification is missing. |
| **Architecture Diagram and Component Design (20%)** | Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented. | Diagram is mostly complete with minor omissions or notation issues. | Diagram includes only essential components and lacks sufficient detail. | Diagram is incomplete, unclear, or technically incorrect. |
| **Component Interaction and Quality Attribute Analysis (15%)** | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps. | Provides limited explanation of interactions and addresses only some quality attributes. | Component interactions and quality attributes are poorly explained or missing. |

| | | | | |
|---|---|---|---|---|
| **Architectural Justification and Technical Presentation (10%)** | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report. | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria | **Mark** |
|---|---|
| System Requirement Analysis **(30%)** | /5 |
| Selection of Software Architecture **(25%)** | /5 |
| Architecture Diagram and Component Design **(20%)** | /5 |
| Component Interaction and Quality Attribute Analysis **(15%)** | /5 |
| Architectural Justification and Technical Presentation **(10%)** | /5 |
| **TOTAL** | /25 |